

Xử lý ambiguity (Handling Pose Ambiguity)

PHỤ THUỘC A/04-pose-ambiguity-target-phang (cơ chế hai nghiệm, quan hệ gương, tỉ số ρ — chương này giả định bạn đã hiểu vì sao có hai nghiệm và chỉ bàn làm gì với chúng); B/09-multi-marker-constellation (cách khử triệt để bằng hình học — chương này là những gì phải làm khi hình học chưa khử hết).

TÓM TẮT

Chương này ngắn, và độ ngắn của nó là có chủ ý: nội dung của nó gần như hoàn toàn là hệ quả của A/04 và B/09. Nó tồn tại riêng vì các quyết định ở tầng code rất cụ thể — ngưỡng bằng bao nhiêu, từ chối hay chấp nhận, chọn nghiệm nào — và trộn chúng vào chương lý thuyết làm cả hai khó dùng. Cấu trúc là một thang bốn bậc theo thứ tự ưu tiên: khử bằng hình học (tốt nhất, và nếu làm được thì ba bậc còn lại thành dự phòng), do và từ chối khi không tin được, dùng nhất quán thời gian có kiểm soát, và fuse với cảm biến khác. Nếu chỉ nhớ một điều: *ba bậc dưới chỉ giúp chọn đúng trong hai nghiệm vẫn đang tồn tại, nên chúng luôn có xác suất sai khác không* — và vì thế chúng không thay thế được bậc trên cùng.

KIẾN THỨC CẦN CÓ TRƯỚC

- **tỉ số ambiguity** ρ — $\rho = e_2/e_1 \geq 1$, tỉ số giữa sai số tái chiếu của nghiệm tốt thứ hai và nghiệm tốt nhất. Gần 1 nghĩa là hai nghiệm giải thích ảnh tốt ngang nhau.
- **từ chối cập nhật** — trả về “không biết” thay vì một pose không đáng tin. Đây là hành vi *đúng* của một hệ đo lường, dù nó làm tăng trên phải xử lý thêm một trạng thái.
- **nất quán thời gian (temporal consistency)** — chọn nghiệm gần với pose ở khung trước, hoặc gần với dự đoán từ mô hình chuyển động. Hiệu quả nhưng có một chế độ hỏng nghiệm trọng mô tả ở mục 4.
- **khoá cứng vào nghiệm sai (lock-in)** — tình trạng mà sau khi chọn nhầm một lần, tiêu chí nhất quán thời gian giữ hệ thống ở nghiệm sai vĩnh viễn — ổn định, tự tin, và sai.

CÂU HỎI MỞ ĐẦU

- Q1. Bạn đã bố trí constellation không đồng phẳng theo B/09. Vì sao chương này vẫn cần thiết, và nó bảo vệ bạn khỏi tình huống nào?
- Q2. Hệ của bạn chọn nghiệm bằng cách lấy cái có sai số tái chiếu nhỏ hơn, và nó chạy ổn định suốt một tuần thử nghiệm. Vì sao sự ổn định ấy có thể là dấu hiệu xấu chứ không phải dấu hiệu tốt?
- Q3. Có một ngưỡng ρ mà dưới nó bạn nên từ chối. Vì sao không thể tra ngưỡng ấy từ một bảng, và bạn dẫn ra nó bằng cách nào?
- Q4. IMU cho biết roll và pitch của robot khá chính xác. Vì sao thông tin ấy *đôi khi* loại được một trong hai nghiệm và đôi khi hoàn toàn vô dụng — điều gì quyết định?
- Q5. Một hệ thống “không bao giờ từ chối” và một hệ thống “thỉnh thoảng nói không biết” — cái nào an toàn hơn cho docking, và vì sao câu trả lời không hiển nhiên với người quản lý dự án?

1.1 Đặt vấn đề

A/04 kết thúc bằng một lời khuyên rõ ràng: khử ambiguity bằng cách phá tính đồng phẳng của constellation, và B/09 mục 4 nói cách làm. Nếu bạn đã làm được điều đó thì phần lớn chương này là dự phòng.

Nhưng “dự phòng” không có nghĩa là không cần, và ba tình huống giải thích vì sao — đây là câu trả lời cho Q1.

Tình huống một: che khuất một phần. Constellation của bạn không đồng phẳng nhờ có tag nghiêng ở hai độ sâu khác nhau. Robot tới gần, một phần constellation ra khỏi khung hình hoặc bị che, và những tag còn lại tình cờ *đồng phẳng*. Bạn quay về bài toán hai nghiệm đúng vào lúc cần chính xác nhất.

Tình huống hai: độ không đồng phẳng chưa đủ. B/09 mục 4B nói rằng ngưỡng độ lệch khỏi mặt phẳng là một câu hỏi định lượng chưa có công thức. Nếu thiết kế của bạn ở gần ngưỡng, thì ở một số tư thế nó đủ và ở một số tư thế khác thì không.

Tình huống ba: giai đoạn xa. Ở cự ly xa, tag nhỏ trên ảnh, và A/04 mục 4 chỉ ra rằng kích thước nhỏ làm ambiguity nặng thêm. Một constellation đủ không đồng phẳng ở 0,5m có thể không đủ ở 3m.

Nói cách khác: khử bằng hình học là *điều kiện vận hành bình thường*, và chương này là *chế độ suy giảm có kiểm soát*. Một hệ thống tốt phải có cả hai, và phải biết mình đang ở chế độ nào.

Có một lý do thứ hai để chương này tồn tại riêng, và nó thuộc về cách tổ chức tri thức hơn là về kỹ thuật. A/04 là chương giải thích cơ chế; nó phải nói về hình học, về phép lật, về vì sao thêm điểm không giúp. Chương này là chương về *quyết định*: ngưỡng đặt ở đâu, làm gì khi vượt ngưỡng, ghi gì vào log. Hai loại nội dung ấy phục vụ hai lúc đọc khác nhau — một lúc học, một lúc viết code — và tách chúng ra làm cả hai dùng được hơn.

1.2 Thang bốn bậc

Cấu trúc của chương, và nó cũng là thứ tự ưu tiên khi thiết kế hệ thống.

Bảng 1.1. Bốn bậc xử lý ambiguity, theo thứ tự ưu tiên giảm dần.

	Biện pháp	Cơ chế	Xác suất sai
1	Khử bằng hình học (B/09)	Phá đồng phẳng $\Rightarrow$ nghiệm thứ hai không còn là cực tiểu	Bằng không
2	Đo ρ và từ chối	Không chọn khi không phân biệt được	Bằng không, nhưng mất dữ liệu
3	Nhất quán thời gian	Chọn nghiệm gần dự đoán	Khác không; có chế độ khoá cứng
4	Fuse cảm biến khác	IMU loại nghiệm sai hướng	Khác không; phụ thuộc hình học

Điều quan trọng nhất trong bảng là cột cuối, và nó là kết luận trung tâm của chương: *chỉ bậc 1 có xác suất sai bằng không*. Bậc 2 không sai nhưng nó không cho câu trả lời. Bậc 3 và 4 cho câu trả lời với một xác suất sai khác không, và xác suất ấy không giảm về không dù cài đặt khéo tới đâu — vì cả hai nghiệm vẫn đang tồn tại và vẫn giải thích ảnh gần như tốt ngang nhau.

Hệ quả thiết kế: *đừng dùng bậc 3 và 4 để thay cho bậc 1*. Dùng chúng để xử lý các tình huống suy giảm ở mục 1, và luôn kèm bậc 2 làm lưới an toàn.

1.3 Bậc 2: đo ρ và từ chối

Bậc rẻ nhất và bậc nên có trong mọi hệ thống. Nó trả lời Q3 và Q5.

A. Lấy ρ ra

Với OpenCV: `solvePnPGeneric` với cờ `SOLVEPNP_IPPE_SQUARE` trả về mảng nghiệm cùng mảng sai số tái chiếu tương ứng. Với thư viện `apriltag` gốc: `estimate_tag_pose` đã tính sẵn cả hai nghiệm và tỉ số của chúng trong `apriltag_pose.c`.

Chi phí gần như bằng không — IPPE tính cả hai nghiệm dù bạn có dùng hay không. Vậy mà rất nhiều cài đặt chỉ lấy nghiệm thứ nhất và vứt phần còn lại. Đây là một trong những cải thiện dễ nhất trong cả cây: *ghi ρ vào log ở mọi khung*, kể cả trước khi bạn quyết định làm gì với nó. Chuỗi ρ theo thời gian tự nó là dữ liệu chẩn đoán giá trị.

B. Ngưỡng đặt ở đâu

Đây là Q3, và câu trả lời trung thực là: *không có hằng số phổ quát, và bạn phải dẫn ra nó cho hệ của mình*.

Lý do không có hằng số: ρ phụ thuộc vào mức nhiễu σ , vào kích thước tag trên ảnh w , vào số điểm, và vào góc nghiêng. Một $\rho = 2$ trên một hệ có $\sigma = 0,05$ px mang nhiều thông tin hơn $\rho = 2$ trên một hệ có $\sigma = 0,3$ px, vì trong trường hợp đầu chênh lệch 2 lần là chênh lệch thật còn trường hợp sau nó có thể là nhiễu.

Cách dẫn ra ngưỡng, và nó là mini project của A/04: chạy Monte Carlo với cấu hình của bạn, ở mỗi lần chạy ghi cả ρ lẫn việc nghiệm được chọn có đúng là nghiệm thật không, rồi vẽ *tỉ lệ chọn sai theo ρ* . Ngưỡng ρ^* của bạn là giá trị mà dưới nó tỉ lệ chọn sai vượt mức bạn chấp nhận được.

Các con số như “ $\rho > 3$ thì tin được” xuất hiện trong tài liệu và tôi đã dùng chúng làm ví dụ ở A/04. *Chúng là ví dụ minh họa, không phải giá trị khuyến nghị đã kiểm* — và tôi ghi rõ điều này thay vì để chúng trở thành một hằng số được chép đi chép lại.

C. Từ chối là hành vi đúng

Đây là Q5, và nó đáng bàn vì nó là một quyết định kiến trúc bị áp lực từ phía ngoài kỹ thuật.

Một hệ thống “không bao giờ từ chối” trông tốt hơn trong demo: nó luôn có câu trả lời, biểu đồ không có lỗ hổng, và không ai phải xử lý trạng thái “không biết”. Một hệ thống “thình thoảng nói không biết” trông kém tin cậy hơn — và đó chính là lý do câu trả lời không hiển nhiên với người quản lý dự án.

Nhưng với docking, hệ thứ hai an toàn hơn nhiều, và lý do là bất đối xứng của hậu quả. Từ chối làm robot chờ thêm vài trăm mili giây hoặc retry — phiền, an toàn, và tự phục hồi. Chọn nhầm nghiệm làm robot lái về một tư thế sai *với sự tự tin đầy đủ*, và với hai nghiệm cách nhau 2α thì sai lệch có thể hàng chục độ. Hậu quả có thể là va chạm.

Nói cách khác: *một phép đo không đáng tin tẻ hơn không có phép đo*, vì không có phép đo thì tăng trên biết mình đang mù, còn một phép đo sai thì nó không biết.

Cách cài đặt đúng: trả về một trạng thái ba giá trị — pose tin cậy, pose không tin cậy kèm lý do, hoặc không có pose — thay vì một pose và một cờ boolean. Tầng điều khiển ở B/13 mục 8 dùng đúng thông tin ấy cho tiêu chí hội tụ.

CẦN NHỚ

Ba thứ phải ghi vào log ở mọi khung, và cả ba đều gần như miễn phí vì chúng đã được tính:

ρ — tỉ số ambiguity. Chuỗi của nó theo thời gian cho biết bạn đang ở vùng an toàn hay vùng nguy hiểm của không gian tư thế.

Nghiệm nào được chọn — nếu nó nhảy qua lại giữa hai nghiệm thì bạn đang ở vùng $\rho \approx 1$ và mọi pose trong đoạn ấy đáng ngờ.

Lý do từ chối khi có. Phân bố các lý do sau một tháng vận hành chỉ ra nút thắt thật của hệ thống.

Ba dòng log ấy biến một hệ thống hộp đen thành một hệ thống chẩn đoán được, và chúng nên có từ ngày đầu chứ không thêm vào khi đã có sự cố.

1.4 Bậc 3: nhất quán thời gian, và chế độ khoá cứng

Bậc này hiệu quả và nguy hiểm, và mục này dành phần lớn cho phần nguy hiểm. Nó trả lời Q2.

Ý tưởng: chọn nghiệm gần với pose ở khung trước, hoặc gần với dự đoán từ odometry bánh xe. Nó hoạt động vì pose thật thay đổi trơn theo thời gian trong khi nghiệm giả nhảy — nên tiêu chí “gần cái trước” thường chọn đúng.

Chế độ hồng đã được nêu ở A/04 mục 5 và đáng lặp lại vì nó là chế độ hồng nguy hiểm nhất trong chương: *một khi đã chọn nhầm một lần, tiêu chí này giữ bạn ở nghiệm sai vĩnh viễn*. Khung tiếp theo, nghiệm sai gần với “pose khung trước” hơn nghiệm đúng, nên nó lại được chọn. Và cứ thế.

Kết quả là một hệ thống *rất ổn định* quanh một giá trị sai. Đây là câu trả lời cho Q2: sự ổn định không phải bằng chứng của tính đúng. Ngược lại, với bài toán ambiguity thì ổn định *quá* có thể là dấu hiệu của khoá cứng — vì một hệ thống trung thực ở vùng $\rho \approx 1$ phải thể hiện sự do dự.

Ba biện pháp phòng, và nên có cả ba.

Một, luôn kết hợp với bậc 2. Nhất quán thời gian chỉ được phép quyết định khi ρ nằm trong một dải trung gian — đủ thấp để không tự tin, đủ cao để không vô vọng. Khi ρ rất thấp, từ chối thay vì dựa vào khung trước.

Hai, kiểm tra lại bằng một nguồn độc lập định kỳ. Ở mỗi vài giây, hoặc mỗi khi ρ tăng lên cao, hãy bỏ qua lịch sử và chọn thuần theo ρ . Nếu kết quả khác với nghiệm đang bám thì bạn vừa phát hiện một khoá cứng.

Ba, ghi log việc chuyển nghiệm. Một hệ thống khoẻ mạnh thỉnh thoảng chuyển nghiệm ở vùng ρ thấp. Một hệ thống *không bao giờ* chuyển trong khi ρ có lúc xuống gần 1 là đáng ngờ.

Ngoài ra có một chi tiết cài đặt quan trọng: khi dùng dự đoán từ odometry, phải nhớ rằng odometry bánh xe có sai số tích lũy và nó tệ nhất ở đúng thứ ta cần — hướng. Với đoạn tiếp cận ngắn thì nó đủ tốt; với một khoảng thời gian dài thì dự đoán trôi và tiêu chí mất hiệu lực.

1.5 Bậc 4: fuse với cảm biến khác

Bậc cuối, và nó trả lời Q4.

IMU cho roll và pitch khá chính xác, vì trọng lực là một tham chiếu tuyệt đối — gia tốc kế đo lực riêng, và khi robot không gia tốc thì nó đo hướng trọng lực. Câu hỏi: thông tin ấy loại được nghiệm sai không?

Câu trả lời là *đôi khi*, và cái quyết định là hình học — đúng như Q4 gợi ý.

Hai nghiệm của target phẳng khác nhau bởi một phép lật quanh một trục nằm trong mặt phẳng tag, và trục ấy vuông góc với mặt phẳng chứa đường nhìn và pháp tuyến tag (A/04 mục 2). Phép lật đổi hướng của camera so với tag, và nó *cũng* đổi hướng camera so với thế giới — nên nó đổi roll và pitch mà camera suy ra.

Nếu trục lật gần *vuông góc* với hướng trọng lực, phép lật đổi roll hoặc pitch đáng kể, và IMU loại được nghiệm sai dễ dàng. Nếu trục lật gần *song song* với hướng trọng lực — tức phép lật chủ yếu đổi yaw — thì roll và pitch gần như không đổi giữa hai nghiệm, và IMU hoàn toàn vô dụng.

Với một robot di động trên sàn phẳng nhìn một tag gắn thẳng đứng trên tường, trục lật thường gần nằm ngang, nên IMU *có* giúp. Nhưng với một tag nghiêng theo hướng khác, tình huống đổi. Kết luận thực hành: *kiểm bằng mô phỏng cho hình học cụ thể của bạn* thay vì giả định nó giúp hay không giúp.

Ghi chú trung thực: tôi dẫn cơ chế này bằng hình học (cửa a). Tôi *chưa* kiểm nó bằng mô phỏng (cửa b) và chưa tìm được nguồn phát biểu tường minh cho trường hợp marker (cửa c). Đây là một mục trong 99-chua-biet.md, và nó kiểm được dễ dàng bằng script của A/04.

Có một biến thể rẻ hơn IMU và đáng nêu: nếu robot chạy trên sàn phẳng và bạn biết chiều cao lắp camera, thì *chính ràng buộc “camera ở độ cao cố định”* loại được nghiệm sai trong nhiều trường hợp. Nghiệm sai thường ngụ ý camera ở một độ cao khác, và kiểm điều đó tốn một phép so sánh. Đây là một ví dụ của nguyên tắc chung: *mọi thông tin tiên nghiệm bạn có về hình học đều là một cơ hội loại nghiệm*, và sàn phẳng là thông tin gần như luôn có với robot di động.

1.6 Giới hạn và chế độ hỏng

Ba bậc dưới không khử được ambiguity. Chúng chọn trong hai nghiệm vẫn đang tồn tại. Xác suất sai của chúng khác không và không giảm về không. Đây là giới hạn cấu trúc, không phải giới hạn cài đặt.

ρ **chỉ có nghĩa khi mô hình đúng.** Sai số tái chiếu tăng lên vì méo chưa khử hoặc constellation sai cũng làm ρ đổi, theo những cách khó diễn giải. Nếu residual tuyệt đối của bạn cao bất thường thì ρ không còn là chỉ báo tin cậy về ambiguity mà là chỉ báo về một vấn đề khác.

Khoá cứng khó phát hiện từ trong dữ liệu. Biện pháp ở mục 4 giúp nhưng không đảm bảo. Cách chắc chắn duy nhất là một nguồn thông tin độc lập — và với docking, nguồn ấy thường là chính cơ khí: nếu robot cập được thì nghiệm đúng, nếu nó trượt thì sai.

Ngưỡng ρ không chuyển được giữa các hệ. Một ngưỡng dẫn ra cho camera này, tag này, khoảng cách này không áp dụng cho cấu hình khác. Phải dẫn lại khi đổi bất cứ thành phần nào.

1.7 Thực hành

Bốn việc, và cả bốn đều rẻ.

Một, lấy cả hai nghiệm và ghi ρ vào log ngay từ ngày đầu, kể cả khi chưa dùng. Dữ liệu ấy không lấy lại được sau này.

Hai, dẫn ra ngưỡng ρ^ bằng Monte Carlo theo mini project của A/04, và dẫn lại khi đổi cấu hình.*

Ba, trả về trạng thái ba giá trị thay vì một pose và một cờ.

Bốn, nếu dùng nhất quán thời gian, hãy cài cả ba biện pháp phòng khoá cứng ở mục 4, đặc biệt là việc định kỳ bỏ qua lịch sử và chọn thuần theo ρ .

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
●●● Vì sao chương này cần dù đã khử bằng hình học	Ba tình huống suy giảm: che khuất làm các tag còn lại tình cờ đồng phẳng; độ không đồng phẳng gần ngưỡng nên đủ ở tư thế này và không đủ ở tư thế khác; và ở cự ly xa tag nhỏ làm ambiguity nặng thêm (A/04 mục 4).
●●● Thang bốn bậc và cột quan trọng nhất	Khử bằng hình học / đo và từ chối / nhất quán thời gian / fuse cảm biến. Cột quan trọng nhất là <i>xác suất sai</i> : chỉ bậc 1 bằng không. Ba bậc dưới chỉ <i>chọn</i> trong hai nghiệm vẫn tồn tại.
●●● Vì sao không có ngưỡng ρ phổ quát	ρ phụ thuộc σ , w , số điểm, góc nghiêng. $\rho = 2$ với $\sigma = 0,05$ px mang nhiều thông tin hơn $\rho = 2$ với $\sigma = 0,3$ px. Phải dẫn ra bằng Monte Carlo cho từng hệ.
●●● Cách dẫn ngưỡng	Chạy Monte Carlo, ghi cả ρ lần việc nghiệm chọn có đúng không, vẽ <i>tỉ lệ chọn sai theo ρ</i> . Ngưỡng là chỗ tỉ lệ sai vượt mức chấp nhận được.
●●● Vì sao từ chối là hành vi đúng	Bất đối xứng hậu quả: từ chối $\Rightarrow$ chờ hoặc retry, an toàn, tự phục hồi. Chọn nhầm $\Rightarrow$ lái về tư thế sai với sự tự tin đầy đủ, lệch có thể hàng chục độ. <i>Một phép đo không đáng tin tệ hơn không có phép đo.</i>
●●● Chế độ khoá cứng	Một khi chọn nhầm, tiêu chí “gắn khung trước” giữ hệ ở nghiệm sai vĩnh viễn — ổn định, tự tin, và sai. Hệ quả chẩn đoán: <i>ổn định không phải bằng chứng của tính đúng</i> .
●●● Ba biện pháp phòng khoá cứng	Chỉ cho nhất quán thời gian quyết định trong dải ρ trung gian; định kỳ <i>bỏ qua lịch sử</i> và chọn thuần theo ρ để phát hiện; và ghi log việc chuyển nghiệm — không bao giờ chuyển trong khi ρ có lúc gần 1 là đáng ngờ.
●● Khi nào IMU loại được nghiệm sai	Khi trục lật gần <i>vuông góc</i> với hướng trọng lực — lúc đó phép lật đổi roll/pitch đáng kể. Khi trục lật gần <i>song song</i> với trọng lực (phép lật chủ yếu đổi yaw) thì IMU vô dụng. (<i>Tôi suy ra từ hình học; chưa kiểm bằng mô phỏng.</i>)
●● Ràng buộc sàn phẳng	Rẻ hơn IMU: nghiệm sai thường ngụ ý camera ở một độ cao khác, và kiểm điều đó tốn một phép so sánh. Ví dụ của nguyên tắc chung — mọi thông tin tiên nghiệm về hình học là một cơ hội loại nghiệm.
●●● Ba thứ phải ghi log mỗi khung	ρ ; nghiệm nào được chọn (nhảy qua lại $\Rightarrow$ đang ở vùng nguy hiểm); và lý do từ chối. Cả ba gần như miễn phí vì đã được tính, và không lấy lại được sau này.
●● Khi nào ρ mất ý nghĩa	Khi residual tuyệt đối cao bất thường — méo chưa khử hoặc constellation sai cũng làm ρ đổi theo cách khó diễn giải. Lúc đó ρ là chỉ báo về một vấn đề khác, không phải về ambiguity.
●● Giao diện trả về đúng	Trạng thái ba giá trị — pose tin cậy / pose không tin cậy kèm lý do / không có pose — thay vì một pose và một cờ boolean. B/13 mục 8 dùng đúng thông tin ấy.

●●● phải thuộc ●● nên nắm ● biết là có

Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
OpenCV solvePnPGeneric	Trả mảng nghiệm kèm mảng sai số tái chiếu $\Rightarrow \rho$ miễn phí	API quan trọng nhất cho chương này
apriltag apriltag_pose.c	estimate_tag_pose tính sẵn cả hai nghiệm và tỉ số	Đọc để thấy bậc 2 đã được cài sẵn trong thư viện gốc
IPPE [1]	Nguồn của việc trả về cả hai nghiệm — một lựa chọn thiết kế API có ý thức	Số bàn chính về việc dùng tỉ số residual làm chỉ báo
S-Tag [2]	Giải quyết ở tầng thiết kế marker thay vì tầng chọn nghiệm	Cùng triết lý với bậc 1: sửa hình học, đừng sửa thuật toán
tagslam [3]	Ràng buộc nhiều tag trong factor graph loại nghiệm không nhất quán	Bậc 1 ở quy mô hệ thống; mức tin C

MINI PROJECT · Dựng một hệ thống ba trạng thái và đo tỉ lệ từ chối trên quỹ đạo thật · 1–2 ngày

Mục tiêu	biến ambiguity từ một rủi ro ẩn thành một đại lượng theo dõi được, và biết chính xác hệ của bạn ở vùng nguy hiểm bao nhiêu phần trăm thời gian.
Dữ liệu	phần cứng thật nếu có, hoặc mô phỏng quỹ đạo tiếp cận đầy đủ nếu chưa.
Các bước	sửa pipeline để lấy cả hai nghiệm và ghi ρ ; dẫn ra ngưỡng ρ^* bằng Monte Carlo (mini project A/04); cài trạng thái ba giá trị; chạy toàn bộ quỹ đạo tiếp cận nhiều lần với các điểm xuất phát khác nhau; vẽ ρ theo khoảng cách Z cho mọi lần chạy, chồng lên nhau; đánh dấu các đoạn $\rho < \rho^*$; và thống kê tỉ lệ phần trăm thời gian ở mỗi trạng thái.
Xong khi	bạn có một biểu đồ ρ theo Z và một con số “hệ thống ở trạng thái không tin cậy $X\%$ thời gian, tập trung ở dải Z từ a tới b ”. Nếu dải ấy trùng với pha cuối thì bạn có một vấn đề thiết kế nghiệm trọng và phải quay lại B/09 mục 4. Kiểm tra chéo bắt buộc: nếu ρ của bạn cao ở mọi tư thế kể cả fronto-parallel với constellation đồng phẳng, thì cách tính ρ của bạn sai — A/04 nói rõ rằng cấu hình ấy phải cho $\rho \approx 1$.
Nối với	B/13 mục 8 dùng trạng thái ba giá trị cho tiêu chí hội tụ; C/16 đưa biểu đồ này vào quy trình nghiệm thu.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì khứ bằng hình học là điều kiện vận hành *bình thường*, không phải điều kiện được đảm bảo ở mọi tư thế. Ba tình huống làm nó mất hiệu lực: che khuất một phần, khi robot tới gần và các tag còn nhìn thấy tình cờ đồng phẳng — đúng vào lúc cần chính xác nhất; độ không đồng phẳng ở gần ngưỡng, nên nó đủ ở tư thế này và không đủ ở tư thế khác; và giai đoạn xa, khi tag nhỏ trên ảnh làm ambiguity nặng thêm ngay cả với constellation tốt. Chương này là chế độ suy giảm có kiểm soát, và một hệ thống tốt phải biết mình đang ở chế độ nào.
- A2. Vì với bài toán ambiguity, ổn định có thể là triệu chứng của *khoá cứng* chứ không phải bằng chứng của tính đúng. Nếu hệ dùng nhất quán thời gian và đã chọn nhầm một lần, thì ở mọi khung sau đó nghiệm sai gần với “pose khung trước” hơn nghiệm đúng, nên nó tiếp tục được chọn — và bạn có một hệ thống rất ổn định quanh một giá trị sai, không có tín hiệu báo động nào. Một hệ trung thực ở vùng $\rho \approx 1$ phải thể hiện sự *do dự*: nó phải thỉnh thoảng chuyển nghiệm. Nên phép thử là: ghi log việc chuyển nghiệm cùng với ρ , và nếu ρ có lúc xuống gần 1 mà hệ không bao giờ chuyển thì đáng ngờ. Biện pháp chắc chắn hơn: định kỳ bỏ qua lịch sử và chọn thuần theo ρ , rồi so với nghiệm đang bám.
- A3. Vì ρ phụ thuộc vào mức nhiễu σ , kích thước tag trên ảnh w , số điểm và góc nghiêng — nên cùng một giá trị ρ mang lượng thông tin khác nhau trên hai hệ khác nhau. Cụ thể: $\rho = 2$ trên một hệ có $\sigma = 0,05$ px nghĩa là chênh lệch thật, còn $\rho = 2$ trên hệ có $\sigma = 0,3$ px có thể chỉ là nhiễu. Cách dẫn ra ngưỡng cho hệ của mình: chạy Monte Carlo với cấu hình thật, ở mỗi lần ghi cả ρ lẫn việc nghiệm được chọn có đúng là nghiệm thật không, rồi vẽ tỉ lệ chọn sai theo ρ ; ngưỡng là chỗ tỉ lệ ấy vượt mức bạn chấp nhận được. Và phải dẫn lại khi đổi bất kỳ thành phần nào của cấu hình.
- A4. Cái quyết định là *hướng của trục lật so với hướng trọng lực*. Hai nghiệm khác nhau bởi một phép lật quanh một trục nằm trong mặt phẳng tag (A/04 mục 2), và phép lật ấy đổi hướng camera so

với thế giới. Nếu trục lật gần vuông góc với hướng trọng lực, phép lật đổi roll hoặc pitch một lượng đáng kể và IMU — vốn đo hướng trọng lực khá chính xác — loại được nghiệm sai dễ dàng. Nếu trục lật gần song song với trọng lực, tức phép lật chủ yếu đổi yaw, thì roll và pitch gần như không đổi giữa hai nghiệm và IMU hoàn toàn vô dụng vì yaw không quan sát được từ trọng lực. Với robot trên sàn nhìn tag thẳng đứng thì trục lật thường gần nằm ngang nên IMU có giúp, nhưng với tag nghiêng thì phải kiểm lại. Có một biến thể rẻ hơn: ràng buộc “camera ở độ cao cố định” trên sàn phẳng cũng loại được nghiệm sai trong nhiều trường hợp, và nó không cần cảm biến nào cả.

- A5. Hệ “thình thoảng nói không biết” an toàn hơn nhiều, vì hậu quả của hai loại lỗi bất đối xứng: từ chối làm robot chờ vài trăm mili giây hoặc retry — phiền, an toàn, và tự phục hồi; còn chọn nhầm nghiệm làm robot lái về một tư thế lệch có thể hàng chục độ với sự tự tin đầy đủ, và hậu quả có thể là va chạm. Nguyên tắc chung: một phép đo không đáng tin tẻ hơn không có phép đo, vì không có phép đo thì tăng trên *biết* mình đang mù. Câu trả lời không hiển nhiên với người quản lý dự án vì hệ không bao giờ từ chối trông tốt hơn trong demo — luôn có câu trả lời, biểu đồ không có lỗ hổng, không ai phải xử lý trạng thái thứ ba. Cái giá của vẻ ngoài ấy chỉ lộ ra khi có sự cố, và lúc đó không có log nào giải thích được vì sao.

NÓI LẠI THẬT ĐƠN GIẢN

Cái tag phẳng có hai câu trả lời và máy không phân biệt được. Chương trước nói cách chữa tận gốc: kê nghiêng mấy tấm tag cho chúng đứng phẳng nữa. Chương này nói phải làm gì trong những lúc cách chữa ấy không đủ — chẳng hạn robot tới gần quá, mấy tấm nghiêng đã ra ngoài khung hình, chỉ còn mấy tấm phẳng ở giữa.

Có bốn mức xử lý, và mức đầu tiên là mức duy nhất thật sự chắc chắn: sửa hình học. Ba mức còn lại chỉ giúp *đoán đúng hơn* giữa hai câu trả lời vẫn đang có, nên chúng luôn có khả năng đoán sai. Mức thứ hai, và nên có trong mọi hệ: *đo xem hai câu trả lời có khớp ảnh ngang nhau tới mức nào*. Máy đã tính sẵn con số đó rồi, chỉ là phần lớn chương trình vớt nó đi. Nếu một câu rõ ràng khớp hơn thì tin; nếu hai câu ngang nhau thì dừng đoán — hãy nói “không biết”.

Nói “không biết” nghe như thất bại, nhưng nó là hành vi đúng, và lý do rất đơn giản: nếu robot không biết thì nó chờ thêm nửa giây hoặc lùi ra thử lại, chẳng mất gì. Còn nếu nó đoán sai thì nó lái đi một hướng lệch hàng chục độ mà vẫn tin chắc mình đúng. Một con số sai tẻ hơn không có con số nào, vì không có con số thì ít ra người ta biết là đang mù.

Mức thứ ba: chọn câu trả lời nào gần với câu trả lời lần trước. Cách này hiệu quả, và nó có một cái bẫy rất xấu. Nếu có một lần đoán nhầm, thì lần sau cái nhầm ấy lại gần với “lần trước” hơn cái đúng, nên nó lại được chọn. Và cứ thế mãi. Kết quả là một hệ thống *rất ổn định* quanh một giá trị sai — không nhảy nữa, không báo lỗi, mọi thứ trông êm đẹp. Nên với bài toán này, ổn định quá lại là dấu hiệu đáng ngờ chứ không phải dấu hiệu tốt. Cách phòng: thỉnh thoảng cố tình quên lịch sử đi và chọn lại từ đầu, rồi xem có ra kết quả khác không.

Mức thứ tư: hỏi một cảm biến khác. Cảm biến quán tính biết khá rõ robot đang nghiêng bao nhiêu so với phương thẳng đứng, và đôi khi thông tin đó loại được một câu trả lời. Nhưng chỉ *đôi khi* — nếu hai câu trả lời chỉ khác nhau ở hướng quay quanh trục đứng, thì cảm biến quán tính không nói được gì, vì nó không biết hướng bắc.

Cuối cùng, một lời khuyên rẻ nhất trong cả chương: *ghi lại mọi thứ vào nhật ký ngay từ ngày đầu*. Con số so khớp của hai câu trả lời, câu nào được chọn, và lý do mỗi lần từ chối. Ba dòng ghi chép đó không tốn gì vì máy đã tính rồi, và một tháng sau chúng là thứ duy nhất cho bạn biết hệ thống thật sự đang gặp chuyện gì.

Chỗ không rút gọn được: ngưỡng “khớp hơn bao nhiêu thì đủ để tin” thì không có con số chung, vì nó phụ thuộc camera, khoảng cách, kích thước tag. Phải tự chạy thử mà tìm, và không chép được từ hệ khác.

Một câu: Ba cách dưới chỉ giúp đoán khéo hơn giữa hai câu trả lời; chỉ có cái nệm nghiêng mới làm biến mất câu trả lời thứ hai.

1.8 Điều gì chứng minh cách hiểu này sai

Mệnh đề chính — *ba bậc dưới có xác suất sai khác không và không thay thế được bậc 1* — là một mệnh đề cấu trúc: nếu hai nghiệm giải thích ảnh tốt ngang nhau thì không thông tin nào *trong ảnh* phân biệt được chúng. Nó bị bác bỏ chỉ nếu ai đó chỉ ra một đại lượng tính từ chính ảnh mà phân biệt được hai nghiệm ở tư thế fronto-parallel — và điều đó mâu thuẫn với A/04.

Mệnh đề thứ hai — *chế độ khoá cứng xảy ra trong thực tế* — bị bác bỏ nếu trên một hệ thực với nhất quán thời gian, việc định kỳ bỏ qua lịch sử không bao giờ phát hiện ra sự bất đồng. Đó sẽ là tin tốt, và nó có thể xảy ra nếu hệ của bạn hiếm khi vào vùng $\rho \approx 1$ — tức nếu B/09 đã làm tốt việc của nó.

Mệnh đề thứ ba — *IMU loại được nghiệm sai khi trục lật vuông góc với trọng lực* — là suy luận hình học chưa kiểm của tôi, đã ghi rõ ở mục 5. Nó bị bác bỏ nếu mô phỏng cho thấy roll và pitch của hai nghiệm gần như luôn giống nhau bất kể hình học. Đây là mệnh đề dễ kiểm nhất trong chương.

1.9 Kết nối

Chương này là chương phụ thuộc nhiều nhất trong cây và nó tự nhận điều đó.

Nối lên trên. A/04-pose-ambiguity-target-phang cấp toàn bộ cơ chế; chương này chỉ bàn quyết định. Mini project của chương ấy là cách dẫn ra ngưỡng ρ^* mà mục 3 cần. A/03-pnp-va-uoc-luong-tu-the cấp solvePnPGeneric và giải thích vì sao khởi tạo bằng IPPE quan trọng.

Nối ngang. B/09-multi-marker-constellation là bậc 1 của thang ở mục 2, và quan hệ giữa hai chương là quan hệ “chính” và “dự phòng”. Nếu B/09 làm tốt thì chương này hiếm khi được kích hoạt — và mini project ở đây chính là cách đo xem B/09 đã làm tốt tới đâu. B/11-uoc-luong-theo-thoi-gian là chương mà mục 4 cảnh báo: mọi kỹ thuật làm mượt theo thời gian chỉ hợp lệ *sau khi* nghiệm đã được chọn tin cậy, và chương này là điều kiện tiên quyết của chương ấy.

Nối xuống. B/13-docking-control mục 8 là chỗ trạng thái ba giá trị ở mục 3C được tiêu thụ: tiêu chí hội tụ đòi “phép đo đáng tin”, và ρ là một trong các thành phần của điều kiện ấy. Và một nỗi đáng nêu: B/13 mục 2 chỉ ra rằng IBVS *không* giải PnP nên không có ambiguity — tức toàn bộ chương này trở nên không liên quan nếu bạn chọn IBVS cho pha cuối. Đây là một trong những lợi ích lớn nhất của IBVS và nó đáng cân nhắc nghiêm túc.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Nêu ba tình huống mà constellation không đồng phẳng vẫn rơi vào bài toán hai nghiệm.
2. Trong bốn bậc xử lý, bậc nào có xác suất sai bằng không, và vì sao ba bậc còn lại không đạt được điều đó dù cài đặt khéo tới đâu?
3. Vì sao không thể tra ngưỡng ρ từ một bảng, và quy trình dẫn ra nó gồm những bước nào?
4. Mô tả chế độ khoá cứng, giải thích vì sao nó làm “ổn định” trở thành dấu hiệu đáng ngờ, và nêu biện pháp phát hiện chắc chắn nhất.
5. Điều gì quyết định việc IMU có loại được nghiệm sai hay không? Nêu một cấu hình mà nó giúp và một cấu hình mà nó vô dụng.
6. Một hệ thống trả về “pose và một cờ boolean” thiếu thông tin gì so với trạng thái ba giá trị, và tầng nào bị thiệt?
7. Vì sao toàn bộ chương này trở nên không liên quan nếu pha cuối dùng IBVS?

Nguồn

- [1] T. Collins and A. Bartoli, “Infinitesimal Plane-Based Pose Estimation,” *International Journal of Computer Vision*, vol. 109, no. 3, pp. 252–286, 2014, mức tin A; IPPE — hai nghiệm cho target phẳng, là nền của SOLVEPNP_IPPE trong OpenCV.
- [2] B. Benligiray, C. Topal, and C. Akinlar, “STag: A Stable Fiducial Marker System,” in *Image and Vision Computing*, mức tin B; dùng đường tròn ngoại tiếp để ổn định pose, vol. 89, 2019, pp. 158–169.
- [3] B. Pfrommer and K. Daniilidis, “TagSLAM: Robust SLAM with Fiducial Markers,” in *arXiv preprint arXiv:1910.00679*, mức tin C; tiền ấn nhưng có mã nguồn công khai và được dùng rộng rãi — dùng làm gợi ý về kiến trúc, không làm bằng chứng số, 2019.